

DEVOPS MASTER COURSE

Volume 9B

Advanced Terraform Engineering

Chapters 11-20

Architecture, testing, policy, HCP Terraform, sensitive data, Stacks, multi-environment delivery, and enterprise operations

2026 Edition

DESIGN

Boundaries and contracts

VERIFY

Tests and policy

OPERATE

Runs, drift, recovery

About Volume 9B

Part B moves from individual Terraform proficiency to platform-scale engineering. It focuses on architecture boundaries, deterministic provider topologies, reusable module contracts, executable tests, policy gates, managed run platforms, secret minimization, multi-environment orchestration, and recovery-driven operations.

The labs remain mostly provider-neutral and use local, random, terraform_data, and test mocking patterns. HCP Terraform and Stacks exercises may be completed as design simulations when an account or paid feature is unavailable. The objective is transferable operating judgment, not product-button memorization.

Operating principle: advanced Terraform is less about writing more HCL and more about constraining change. Stable interfaces, isolated state, tested assumptions, policy gates, short-lived identity, serialized execution, and rehearsed recovery reduce infrastructure risk.

Part B Chapter Map

Chapter	Focus	Engineering Outcome
11	Architecture boundaries and state decomposition	Choose ownership and blast-radius boundaries deliberately.
12	Advanced HCL, dynamic structures, and conditions	Write expressive configuration without obscuring intent.
13	Provider aliases and multi-account or multi-region topology	Make credentials and target contexts explicit.
14	Advanced module contracts and registries	Publish reusable modules with compatibility discipline.
15	Terraform tests, mocking, checks, and conditions	Verify modules before and during infrastructure changes.
16	Policy as code and governance gates	Turn non-negotiable requirements into tested controls.
17	HCP Terraform and Terraform Enterprise operations	Operate projects, workspaces, runs, agents, and access.
18	Sensitive data, ephemeral values, and write-only arguments	Reduce secret persistence in plans and state.
19	Terraform Stacks and multi-environment orchestration	Deploy reusable components consistently at scale.
20	Enterprise operations and final Part B capstone	Integrate design, testing, policy, delivery, drift, and recovery.

Lab Safety and Evidence Standard

- Use disposable directories and isolated state. Do not run examples against production subscriptions, accounts, clusters, or SaaS tenants.

- Pin Terraform and provider versions. Record the exact versions used for every test or incident reproduction.
- Treat saved plans, state snapshots, test fixtures, policy input, debug logs, and run artifacts as potentially sensitive.
- For each lab, preserve the reviewed configuration, command transcript, plan summary, assertions, policy result, validation result, and cleanup proof.
- When a hosted feature is unavailable, produce a faithful design simulation with trust boundaries, roles, run stages, failure paths, and validation criteria.
- Never force-unlock, edit state, bypass policy, or approve a replacement merely to finish a lab. Diagnose and document the safe path.

Chapter 11: Terraform Architecture Boundaries and State Decomposition

Enterprise Terraform systems fail when state boundaries follow repository convenience instead of ownership, lifecycle, sensitivity, and blast radius. Architecture starts by deciding what changes together, who may change it, and how downstream consumers receive contracts.

Learning Objectives

- Evaluate monolithic, layered, and domain-oriented Terraform architectures.
- Use coupling, ownership, sensitivity, and change frequency to define state boundaries.
- Publish outputs through explicit interfaces instead of broad state access.
- Design dependency and failure containment across multiple root modules.

Core Concepts

State boundary: A root module and its state form an operational transaction boundary. Resources that must change atomically often belong together; unrelated domains should not share a failure surface.

Ownership boundary: Each state has a named team, escalation path, deployment identity, maintenance window, and recovery procedure.

Dependency contract: Consumers should receive the smallest stable interface required, preferably through a purpose-built registry, parameter store, DNS, or data source rather than unrestricted state access.

Layering: Foundation, platform, shared-service, and workload layers can reduce blast radius, but excessive layering creates orchestration and compatibility overhead.

Change frequency: Fast-changing application infrastructure should not be locked behind slow-moving foundation state unless atomicity truly requires it.

Failure containment: A provider outage, lock, invalid policy, or partial apply in one state should not freeze unrelated business domains.

Decision Signal	Keep Together	Separate
Lifecycle	Must be created and destroyed as one unit	Independent lifecycle or release cadence
Ownership	Same accountable team and approver	Different teams or privilege domains
Sensitivity	Same confidentiality and access model	Different secret, compliance, or tenant boundary
Failure impact	Atomic recovery required	Independent recovery reduces blast radius
Dependency	Tight bidirectional coupling	Stable output contract is sufficient

Reference Configuration or Command Pattern

```
# Producer root module
output "service_contract" {
  value = {
    endpoint = local.endpoint
    version  = "v2"
  }
}

# Consumer receives a narrow contract through an approved mechanism.
variable "service_contract" {
  type = object({
    endpoint = string
    version  = string
  })

  validation {
    condition     = var.service_contract.version == "v2"
    error_message = "Unsupported service contract version."
  }
}
```

Production Checklist

- Every state has a named owner, deployment identity, backend, lock model, backup policy, and recovery objective.
- Cross-state interfaces are versioned and intentionally narrow.
- State access is not used as a substitute for service discovery or secret distribution.
- Dependency direction is acyclic or controlled through an explicit orchestration layer.
- Architecture diagrams show state, identity, network, policy, and data-flow boundaries.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
One plan contains hundreds of unrelated changes	Monolithic state and broad provider scope	Pause rollout; inventory ownership and coupling; design staged decomposition	Each new state produces focused plans and independent recovery
Consumer breaks after producer output change	Unversioned cross-state contract	Restore compatibility; version outputs and test consumers	Consumer test passes against old and new contract
State split proposes recreation	Bindings moved without migration mapping	Back up state; use import, moved blocks, or state mv with peer review	No unintended create or destroy remains
Remote-state reader sees excessive data	Broad backend access	Replace with narrow published contract and least privilege	Consumer can read only required values

Hands-on Lab 11: Decompose a Monolithic Terraform System

Prerequisites

- Completed Volume 9A.
- A disposable Terraform project using `local_file` or `terraform_data` resources.
- A diagramming tool or plain-text architecture format.

Implementation

1. Create a monolithic root that manages foundation, shared-service, and application artifacts in one state.
2. Document owners, release cadence, sensitivity, dependencies, and failure impact for each artifact.
3. Design three target root modules and versioned contracts between them.
4. Back up state and migrate bindings without recreation using moved blocks, imports, or controlled state moves.
5. Run plans for all roots and verify that each plan is focused and no resource is orphaned.
6. Simulate a locked or failing application root and prove the foundation root remains operable.

Validation Evidence

- Before-and-after state architecture diagrams are complete.
- No unintended resource replacement occurs during decomposition.
- Each root has isolated state, explicit ownership, and a recovery procedure.
- Consumer contracts are versioned and validated.

Cleanup

- Destroy all managed local artifacts.
- Remove temporary state backups and saved plans after recording sanitized evidence.
- Retain the architecture decision record and migration runbook.

Interview Questions and Model Answers

1. What is the most important Terraform architecture boundary?

The state and execution boundary, because it defines ownership, concurrency, blast radius, credentials, and recovery.

2. Why not create one state per resource?

Over-fragmentation creates orchestration, dependency, access, and operational overhead without meaningful failure isolation.

3. Why is `terraform_remote_state` risky as a default integration method?

The consumer generally needs access to the source state snapshot even when it only requires a few outputs. A narrower publication mechanism can reduce exposure and coupling.

4. How do you safely split state?

Inventory bindings, freeze concurrent writes, back up state, create target configuration, migrate or import exact addresses, compare plans, validate functionality, and retain rollback evidence.

Chapter Summary

- State decomposition is an architecture and operating-model decision.
- Boundaries should follow ownership, lifecycle, sensitivity, coupling, and recovery.
- Cross-state contracts must be narrow and versioned.
- Migration is complete only when plans, functionality, access, and recovery are validated.

Chapter 12: Advanced HCL, Dynamic Structures, and Runtime Conditions

Advanced expressions can remove duplication, but clever HCL can become an unreadable programming language hidden inside infrastructure code. The goal is expressive, typed, reviewable configuration with failures close to the source.

Learning Objectives

- Use complex types, for expressions, splats, merge, flatten, setproduct, and try safely.
- Generate repeated nested blocks with dynamic only when the schema requires it.
- Apply variable validation, preconditions, postconditions, and check blocks appropriately.
- Recognize when preprocessing or a module redesign is clearer than advanced HCL.

Core Concepts

Type-first design: Define object shapes before writing transformations. Strong types make module contracts, tests, and refactors safer.

Normalization locals: Convert flexible external input into one canonical internal structure near the module boundary.

For expressions: Transform collections explicitly and select stable keys. Avoid index-based identity for long-lived resources.

Dynamic blocks: Use dynamic only for repeatable nested blocks required by a provider schema; it cannot generate meta-arguments.

Defensive functions: try and can can normalize optional input, but broad error suppression can hide defects. Keep fallback logic local and test it.

Conditions: Validation rejects bad inputs; preconditions protect assumptions before an operation; postconditions verify resulting attributes; check blocks continuously evaluate broader assertions without necessarily blocking every operation.

Construct	Best Use	Common Failure
for expression	Transform or filter a collection	Unstable or duplicate keys
flatten	Normalize nested lists	Losing parent context
merge	Compose maps with precedence	Unexpected overwrite order
try/can	Handle known optional shapes	Masking genuine errors
dynamic	Emit repeated nested provider blocks	Over-abstracted unreadable configuration
condition/check	Enforce invariant or health assertion	Using warning-style checks for mandatory safety

Reference Configuration or Command Pattern

```
variable "services" {
  type = map(object({
    port  = number
    enabled = optional(bool, true)
    tags  = optional(set(string), [])
  }))

  validation {
    condition = alltrue([for s in values(var.services) : s.port >= 1 && s.port <= 65535])
    error_message = "Every service port must be between 1 and 65535."
  }
}

locals {
  enabled_services = {
    for name, svc in var.services : name => merge(svc, {
      tags = setunion(svc.tags, ["managed-by-terraform"])
    }) if svc.enabled
  }
}
```

Production Checklist

- Inputs have explicit types and validation.
- Collection keys represent durable identity.
- Normalization occurs once, near the boundary.
- Dynamic blocks are documented with an example of expanded output.
- Critical safety invariants block the run; informational health checks are routed to owners.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
for_each reports duplicate key	Transformation generated non-unique identity	Select a durable composite key and add validation	Plan shows one address per intended object
try returns fallback unexpectedly	Input shape differs from contract	Inspect value in terraform console; tighten type and tests	Expected branch is selected for all fixtures
Plan changes many addresses	List ordering or count index changed	Migrate to stable map keys with moved blocks	Plan has no unintended replacement
Check warns forever	No owner or remediation workflow	Assign severity, owner, and alert or convert to blocking condition	Check has documented response and passes

Hands-on Lab 12: Build a Typed Configuration Normalization Layer

Prerequisites

- Terraform CLI and terraform console.
- A local module with no cloud dependencies.
- Three input fixtures: valid, optional fields omitted, and invalid.

Implementation

1. Define a map(object) variable for services with optional attributes and validation.
2. Create locals that normalize tags, defaults, and enabled-service filtering.
3. Use for_each on terraform_data resources keyed by service name.
4. Add a precondition that prevents a reserved port and a check block that verifies at least one service is enabled.
5. Use terraform console to inspect intermediate expressions.
6. Create tests for valid, defaulted, invalid, and empty configurations.

Validation Evidence

- All valid fixtures produce stable addresses.
- Invalid ports fail at variable validation.
- Reserved ports fail before resource operation.
- The empty configuration produces the intended check result.

Cleanup

- Destroy or remove terraform_data resources.
- Delete test-generated state and temporary plan files.
- Retain sanitized fixtures and expression notes.

Interview Questions and Model Answers

1. When should dynamic blocks be used?

When a provider resource schema contains a repeatable nested block and a clear data structure must generate it. They should not be used merely to appear generic.

2. Why prefer stable map keys over list indexes?

Map keys preserve resource identity when unrelated elements are inserted, removed, or reordered.

3. What is the difference between validation and precondition?

Variable validation checks an input value at the variable boundary. A precondition checks an assumption in the context of a resource, data source, or output before the relevant operation.

4. What is a check block for?

It expresses an assertion about infrastructure or data that Terraform can evaluate during normal operations and continuous validation workflows, often to detect degraded conditions outside a single resource lifecycle.

Chapter Summary

- Advanced HCL should reduce duplication without reducing comprehension.
- Normalize flexible input into a strict internal model.
- Stable keys protect resource identity.
- Place conditions where they produce the earliest and clearest failure.

Chapter 13: Provider Aliases and Multi-Account, Multi-Region Topologies

Provider configuration is an identity and targeting decision. Hidden defaults, implicit inheritance, or broad credentials can direct a correct resource block to the wrong account, subscription, project, region, cluster, or SaaS tenant.

Learning Objectives

- Use provider aliases and explicit module provider mappings.
- Design credential and provider boundaries for multiple targets.
- Prevent accidental use of an empty or default provider configuration.
- Test provider topology and failure modes without exposing credentials.

Core Concepts

Provider instance: A provider source can have multiple configured instances, each with its own alias, target settings, and authentication context.

Default provider: Resources without a provider meta-argument use the default instance. If all provider blocks are aliased, Terraform may create an implied empty default that fails later.

Module mapping: Child modules declare configuration aliases they can accept; root modules explicitly map provider instances into each module call.

Credential scope: Use one short-lived, least-privilege identity per trust boundary where practical. Do not make a single super-credential the convenience default.

Target metadata: Region, account, subscription, project, endpoint, and tenant should be visible in provider construction and run metadata.

Testing: Mock providers and contract tests can verify alias wiring, while live integration tests verify target-specific API behavior in disposable environments.

Topology	Advantage	Risk and Control
One root, multiple aliases	Atomic cross-target change	Large blast radius; strong approval and rollback required
One root per target	Clear state and identity isolation	Orchestration overhead; use shared modules and promotion
Hub-and-spoke providers	Central control with delegated targets	Privilege concentration; constrain mappings and audit
Dynamic credentials	Short-lived and attributable	Token lifecycle; test refresh and expiry behavior
Static credentials	Simple bootstrap	Rotation and leakage risk; avoid for production

Reference Configuration or Command Pattern

```
terraform {
  required_providers {
    example = {
      source = "example/example"
      configuration_aliases = [example.primary, example.secondary]
    }
  }
}

# Root module maps explicit provider instances.
module "replicated_service" {
  source = "../modules/replicated-service"
  providers = {
    example.primary = example.region_a
    example.secondary = example.region_b
  }
}
```

Production Checklist

- Every provider instance has an explicit purpose, target, credential source, and owner.
- Child modules do not embed environment credentials or hard-coded target identifiers.
- Provider mappings are explicit at module boundaries.
- Default-provider behavior is tested and documented.
- Cross-target changes have failure sequencing, compensation, and validation plans.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Resource appears in wrong region	Implicit default provider selected	Stop; identify provider address and state binding; correct mapping safely	Plan and remote inventory show intended target
Child module says provider not configured	configuration_aliases or mapping missing	Declare accepted aliases and map them from root	terraform validate and test pass
All providers aliased but resource has none	Implied empty default configuration	Add explicit provider meta-argument or intentional default	No resource relies on accidental default
Token expires during long apply	Credential TTL shorter than operation	Use supported refresh/federation or reduce operation scope	Long-running integration test completes

Hands-on Lab 13: Create and Test a Multi-Provider Module Contract

Prerequisites

- A mockable or local provider pattern.
- Terraform test support.
- No production credentials.

Implementation

1. Create a child module that requires primary and secondary provider aliases.
2. Create root provider instances with explicit aliases and map them into the module.
3. Add tests with mock providers that assert outputs identify the intended primary and secondary contexts.
4. Remove one mapping and confirm validation or test failure is clear.
5. Add a resource without a provider meta-argument and demonstrate why an unintended default is dangerous.
6. Document the target, identity, and rollback model for a real two-region deployment.

Validation Evidence

- The module accepts only declared aliases.
- Tests prove primary and secondary wiring.
- Missing or incorrect mappings fail before production apply.
- No credential is stored in code, state, or test fixture.

Cleanup

- Remove generated test artifacts and local state.
- Revoke any temporary test identity.
- Retain provider-topology diagram and contract documentation.

Interview Questions and Model Answers

1. What does a provider alias represent?

A named provider configuration instance with its own target and authentication settings.

2. Do child modules inherit aliased providers automatically?

No. Aliased configurations must be declared by the child and explicitly passed in the module providers map.

3. Why is a default provider risky in multi-account systems?

A resource that omits its provider can silently use the default target, causing deployment to the wrong trust boundary.

4. When should multi-region infrastructure be one state?

Only when the lifecycle must be coordinated atomically and the increased blast radius, credentials, failure modes, and recovery can be controlled.

Chapter Summary

- Provider configuration is target and identity architecture.
- Aliases must be explicit at module boundaries.
- Avoid broad convenience defaults.
- Test wiring separately from live API integration.

Chapter 14: Advanced Module Engineering, Registries, and Compatibility

Modules become products when many teams depend on them. At that point, interface stability, semantic versioning, examples, tests, documentation, security ownership, release evidence, and deprecation policy matter more than internal cleverness.

Learning Objectives

- Design narrow, typed module interfaces and stable outputs.
- Apply semantic versioning and compatibility rules.
- Publish modules to a registry with provenance and examples.
- Operate module upgrades, deprecations, and security fixes safely.

Core Concepts

Module contract: Inputs, outputs, required providers, behavior, invariants, and side effects form the supported interface.

Composition: Prefer small modules composed by root modules over deeply nested abstraction pyramids. Every layer must add a meaningful policy or reusable capability.

Versioning: Patch releases fix compatible defects, minor releases add compatible capability, and major releases permit breaking contracts. Document exceptions explicitly.

Release artifact: A module release should be immutable, tagged, tested, documented, and traceable to a reviewed commit.

Upgrade path: Provide migration notes, moved blocks where applicable, changed defaults, expected plan effects, and rollback guidance.

Registry operation: Private registries centralize discovery and versioning, but access, namespace ownership, provenance, and retirement require governance.

Contract Area	Compatibility Question	Required Evidence
Input	Does type, default, validation, or meaning change?	Old and new fixtures
Output	Is name, type, sensitivity, or semantic meaning stable?	Consumer contract tests
Resource address	Will an upgrade replace or orphan resources?	Before/after plan and moved mapping
Provider	Are source, version, or alias requirements compatible?	Lock and integration matrix
Behavior	Does the same input produce materially different infrastructure?	Release notes and acceptance tests

Reference Configuration or Command Pattern

```
# versions.tf in a reusable module
terraform {
  required_version = ">= 1.10, < 2.0"
  required_providers {
    random = {
      source = "hashicorp/random"
      version = ">= 3.6, < 4.0"
    }
  }
}

variable "name" {
  type      = string
  description = "Stable logical name used by consumers."
  validation {
    condition = can(regex("^[a-z][a-z0-9-]{2,30}$", var.name))
    error_message = "name must be a lowercase identifier."
  }
}
```

Production Checklist

- Module source and releases are immutable and traceable.
- Inputs and outputs are typed, described, validated, and tested.
- Examples are executable and represent supported usage.
- Provider constraints balance compatibility with safety; root modules control final selections.
- Deprecations include dates, migration instructions, and an accountable owner.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Minor upgrade proposes replacement	Behavior or address changed incompatibly	Stop release; restore compatibility or publish major version with migration	Upgrade plan is documented and accepted
Consumers pin to commits	No release discipline or registry	Publish semantic tags and registry versions; enforce allowed sources	Consumer versions are auditable
Module hides provider configuration	Credentials or target embedded internally	Move provider configuration to root and declare aliases	Module is portable and testable
Security fix cannot reach consumers	No inventory or upgrade campaign	Inventory versions; publish advisory and deadline; track adoption	Affected versions reach approved target

Hands-on Lab 14: Publish a Versioned Module Release

Prerequisites

- A Git repository.
- Terraform CLI and terraform test.

- A local or private-registry simulation.

Implementation

1. Create a reusable naming or service-metadata module with typed inputs, outputs, examples, and tests.
2. Release version 1.0.0 and consume it from two example roots.
3. Add a backward-compatible optional feature and release 1.1.0.
4. Create a deliberately breaking input rename, then redesign it with a compatibility bridge or release 2.0.0 with migration guidance.
5. Generate before-and-after plans for each consumer.
6. Write SUPPORT.md with ownership, supported versions, security response, and retirement policy.

Validation Evidence

- Both consumers pass tests on supported releases.
- The minor upgrade has no breaking plan effect.
- The major migration contains explicit instructions and expected plan changes.
- Release tags and changelog map to reviewed commits.

Cleanup

- Destroy test resources and remove temporary state.
- Delete unpublished experimental tags.
- Retain sanitized release evidence and migration guide.

Interview Questions and Model Answers

1. What makes a module production-ready?

A clear contract, typed validation, examples, automated tests, documentation, versioned immutable releases, ownership, security response, and upgrade guidance.

2. Who should configure providers?

Root modules. Reusable child modules declare requirements and aliases but should not embed environment credentials or targets.

3. Why avoid deep module nesting?

It obscures ownership, provider flow, resource addresses, errors, and plan reasoning while making upgrades harder.

4. How do you decide a major version is required?

When consumers must change configuration or behavior to upgrade safely, or when resource identity and outputs are not backward-compatible.

Chapter Summary

- Reusable modules are maintained products.
- Compatibility includes behavior and resource identity, not only syntax.
- Root modules own providers and environment composition.
- Every release needs tests, evidence, ownership, and migration guidance.

Chapter 15: Terraform Tests, Mocking, Checks, and Contract Verification

Formatting and validation prove only that configuration is syntactically and internally coherent. Production modules need executable evidence for inputs, plans, outputs, invariants, provider interactions, upgrades, and failure behavior.

Learning Objectives

- Write .tftest.hcl files with plan and apply run blocks.
- Use assertions, expected failures, mocks, and overrides.
- Separate unit-style contract tests from live integration tests.
- Design upgrade and destructive-change tests in CI.

Core Concepts

Test file: Terraform test files define run blocks, variables, providers, assertions, expected failures, and optional mocks or overrides.

Plan test: A plan run verifies configuration and planned values without creating infrastructure. It is fast and suitable for many contract checks.

Apply test: An apply run creates temporary test infrastructure and can verify computed results. It requires isolated credentials, budget, cleanup, and timeout controls.

Mock provider: Provider mocking and overrides simulate computed values without calling the remote API, improving speed and determinism.

Expected failure: Negative tests prove invalid input or conditions fail at the intended boundary with a useful message.

Test pyramid: Use many local contract tests, fewer disposable integration tests, and a small number of end-to-end platform acceptance tests.

Test Layer	Purpose	Typical Gate
Static	fmt, validate, lint, schema checks	Every commit
Contract	Inputs, outputs, addresses, conditions, mocks	Every pull request
Integration	Real provider behavior in disposable target	Merge or scheduled
Upgrade	Old state/configuration upgraded to candidate version	Release candidate
Policy	Plan complies with mandatory controls	Before apply
Acceptance	Service-level user journey and operations	Promotion

Reference Configuration or Command Pattern

```
# tests/module.tftest.hcl
mock_provider "random" {}

run "valid_name_plan" {
  command = plan

  variables {
    name = "payments-api"
  }

  assert {
    condition = output.normalized_name == "payments-api"
    error_message = "Module changed the stable output contract."
  }
}

run "invalid_name_rejected" {
  command = plan
  variables { name = "BAD NAME" }
  expect_failures = [var.name]
}
```

Production Checklist

- Tests run with pinned Terraform and provider versions.
- Live tests use disposable isolated targets and short-lived identity.
- Assertions verify behavior, not internal implementation details unless identity stability is required.
- Negative, upgrade, timeout, cleanup, and partial-failure paths are tested.
- Test logs and state are protected, retained only as needed, and cleaned automatically.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Mock test passes but live test fails	Provider/API behavior not represented by mock	Add targeted integration test and improve contract assumptions	Both mock and live suites pass
Apply test leaks resources	Cleanup failure or interrupted run	Use unique test scope, TTL tagging, janitor job, and manual runbook	Inventory shows zero orphaned resources
Assertion depends on unknown value	Plan cannot compute provider result	Use apply test, override, or assert a known structural property	Test evaluates deterministically
Upgrade test replaces resource	Address or provider behavior changed	Add moved mapping or publish breaking release with migration	Upgrade plan matches release contract

Hands-on Lab 15: Build a Terraform Test Pyramid

Prerequisites

- A reusable module from Chapter 14.
- Terraform version supporting terraform test and provider mocks.
- Optional disposable provider target for one live test.

Implementation

1. Create plan tests for defaults, optional inputs, outputs, and stable resource keys.
2. Add negative tests for validation and preconditions using `expect_failures`.
3. Mock a provider and override one computed resource value.
4. Add one apply-style test using only local or disposable resources.
5. Create an upgrade fixture from the previous module release and compare plan behavior.
6. Integrate `fmt`, `validate`, `terraform test`, and test cleanup into a CI script.

Validation Evidence

- All contract and negative tests produce deterministic results.
- Mocked values are explicit and do not conceal unsupported assumptions.
- The apply test cleans up automatically.
- Upgrade tests prove the documented compatibility level.

Cleanup

- Run `terraform test cleanup` or destroy remaining test state.
- Delete plan, state, and debug artifacts.
- Retain sanitized CI logs and a test coverage matrix.

Interview Questions and Model Answers

1. What does terraform validate not test?

It does not prove remote API behavior, policy compliance, functional service behavior, upgrade safety, or runtime values.

2. When should a test use `command = apply`?

When assertions depend on computed provider results or when real creation behavior must be verified in an isolated disposable environment.

3. Why use provider mocking?

To test module logic and contracts quickly and deterministically without creating remote infrastructure.

4. What is an upgrade test?

A test that starts from a prior supported configuration or state and verifies the candidate version produces only documented, acceptable changes.

Chapter Summary

- Terraform tests convert module promises into executable evidence.
- Mocks accelerate contract testing but do not replace live integration.
- Negative and upgrade tests are essential.
- Test infrastructure requires the same identity, cost, cleanup, and evidence controls as production automation.

Chapter 16: Policy as Code and Governance Gates

Policy as code transforms architectural and compliance requirements into consistent, reviewable decisions. Poor policy programs create bypass culture; effective programs define severity, ownership, evidence, exceptions, tests, and remediation before enforcement.

Learning Objectives

- Design preventive and advisory Terraform policy controls.
- Explain Sentinel policy sets and alternative plan-policy engines.
- Test policies with representative pass, fail, unknown, and exception cases.
- Operate policy exceptions without normalizing permanent bypass.

Core Concepts

Policy input: Policies may evaluate configuration, plan changes, provider schemas, state, cost estimates, metadata, and run context. The input must be versioned and protected.

Enforcement level: Advisory policies inform; soft mandatory policies permit authorized override; hard mandatory policies block. Use the weakest level that still controls the risk.

Policy set: Group related controls, ownership, scope, and version together. Attach sets to projects, workspaces, or run pipelines deliberately.

Exception: An exception requires scope, reason, risk acceptance, approver, compensating control, expiry, and review. It is not an undocumented override click.

Testing: Policy tests require passing, failing, boundary, unknown-value, and exception fixtures. False positives are operational defects.

Remediation: A policy should explain what failed, why it matters, where in the plan it occurred, and how to correct it.

Policy Class	Example	Enforcement Strategy
Security	Public exposure or unencrypted storage	Hard mandatory after tested rollout
Reliability	No redundancy or backup	Soft mandatory during adoption, then hard
Cost	Oversized instance or absent budget tag	Advisory plus approval threshold
Governance	Missing owner, environment, or data class	Hard for new resources; migration campaign for old
Change safety	Destroy or replacement of protected class	Mandatory human approval and backup evidence

Reference Configuration or Command Pattern

```
# Sentinel-style policy intent (illustrative)
import "tfplan/v2" as tfplan

mandatory_tags = ["owner", "environment", "data_class"]

main = rule {
  all tfplan.resource_changes as _, rc {
    rc.change.actions is ["delete"] or
    all mandatory_tags as tag {
      rc.change.after.tags contains tag
    }
  }
}
```

Production policies also need mocks, clear messages,
exception metadata, and scoped rollout evidence.

Production Checklist

- Each policy has an owner, rationale, severity, scope, tests, and remediation text.
- Policy version and input schema are pinned to the run.
- Exceptions expire automatically and are reported.
- Unknown values are handled explicitly rather than silently allowed or denied.
- Policy metrics track failure rate, override rate, false positives, and remediation time.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Policy blocks valid plan	False positive or unsupported resource shape	Pause enforcement; reproduce with fixture; fix and regression-test	Valid fixture passes without weakening control
Teams repeatedly override	Policy misfit or exception process too easy	Analyze causes; improve remediation; tighten override authority	Override rate and age decline
Unknown values bypass control	Policy treats unknown as compliant	Add conservative handling or post-plan/run-task validation	Unknown fixture produces intended decision
Policy update breaks all runs	Unversioned central change	Pin/release policy sets; canary by project; rollback quickly	Canary and rollback tests pass

Hands-on Lab 16: Create and Test a Terraform Policy Pack

Prerequisites

- A plan JSON fixture or HCP Terraform policy simulation.
- Sentinel CLI or another approved policy engine.
- No production plan data.

Implementation

1. Define policies for mandatory ownership metadata, restricted destructive actions, and approved module sources.
2. Create pass, fail, boundary, unknown, and temporary-exception fixtures for each policy.
3. Write policy tests and run them locally or in CI.
4. Define advisory, soft mandatory, and hard mandatory rollout stages.
5. Create an exception record with owner, reason, compensating control, expiry, and review.
6. Produce a run report that identifies failed resources and remediation guidance.

Validation Evidence

- All policy fixtures return expected results.
- Messages identify exact failing controls and remediation.
- Exception expires and cannot become an unbounded bypass.
- The rollout plan contains canary scope and rollback.

Cleanup

- Remove any uploaded real plan data.
- Delete temporary policy tokens and test artifacts.
- Retain sanitized policy code, tests, and governance runbook.

Interview Questions and Model Answers

1. What makes a good infrastructure policy?

It is specific, testable, scoped, understandable, owned, measurable, and connected to remediation and exception processes.

2. When should a policy be advisory?

During discovery, when the risk is informational, or when false-positive and remediation data must be gathered before blocking.

3. How should unknown plan values be handled?

Explicitly. Depending on the risk, defer to a later stage, require evidence, or fail conservatively rather than assuming compliance.

4. What is the danger of broad soft-mandatory override access?

It converts mandatory policy into an optional warning and weakens accountability.

Chapter Summary

- Policy as code needs product-level testing and operations.
- Enforcement must match risk and adoption maturity.
- Exceptions are scoped, approved, compensating, and expiring.
- Measure false positives, overrides, and remediation time.

Chapter 17: HCP Terraform and Terraform Enterprise Operations

A managed Terraform run platform centralizes state, execution, identity, policy, access, audit, and integrations. It improves control only when projects, workspaces, teams, variable sets, agents, run tasks, and recovery procedures are deliberately designed.

Learning Objectives

- Explain projects, workspaces, runs, execution modes, and state in HCP Terraform.
- Design VCS, CLI, API, and agent-based workflows.
- Apply team permissions and variable sets with least privilege.
- Operate run tasks, drift detection, concurrency, and incident recovery.

Core Concepts

Project: Projects group workspaces and Stacks and provide an access and organizational boundary.

Workspace: A workspace combines configuration source, variables, state, run history, permissions, and execution settings for an infrastructure boundary.

Run workflow: Runs move through fetch, plan, cost or task checks, policy, approval, apply, and post-apply stages depending on configuration.

Execution mode: Remote mode uses managed workers; agent mode executes through self-hosted agents for private networks; local mode uses HCP Terraform state with local execution.

Variable set: Variable sets distribute configuration or credentials across scoped projects and workspaces. Scope and precedence must be documented.

Agent pool: Agents poll outward and execute in controlled private environments. Pools require token protection, patching, capacity, observability, and isolation.

Capability	Design Question	Failure Control
Projects	Does grouping match ownership and access?	Quarterly access and orphan review
Workspaces	Is state boundary and trigger unambiguous?	Naming, tags, lock, run queue, owner
VCS runs	Which branch/path triggers?	Speculative plan and protected merge
Agents	Which network and secrets are reachable?	Scoped pools, no ingress, rotation, capacity
Run tasks	What external decision can block?	Timeout, fail mode, authentication, evidence
State	How is recovery performed?	Version history, access audit, restore procedure

Reference Configuration or Command Pattern

```
terraform {
  cloud {
    organization = "example-org"
    workspaces {
      name = "payments-prod"
    }
  }
}
```

Do not place tokens in configuration.
 # Use the supported CLI credential mechanism or workload identity.
 # Workspace variables and variable sets require explicit scope review.

Production Checklist

- Projects and workspaces map to accountable teams and state boundaries.
- Production apply permission is separate from plan and read access.
- VCS branches, working directories, triggers, and speculative plans are tested.
- Agent pools are isolated, monitored, patched, capacity-tested, and token-rotated.
- Run tasks and policy checks define timeout, availability, authentication, and fail-open or fail-closed behavior.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Workspace never triggers	VCS branch, path, or webhook configuration wrong	Inspect integration status and trigger settings	Test commit creates speculative or normal run
Run queued indefinitely	Concurrency, agent capacity, or prior apply lock	Identify blocking run and agent health; scale safely	Run starts with one active writer
Agent cannot reach provider API	Network, DNS, proxy, certificate, or credential issue	Test from agent namespace; inspect logs and egress policy	Controlled diagnostic succeeds
Variable has unexpected value	Variable set precedence or duplicate key	Inventory all scopes and categories; remove ambiguity	Run input report matches design

Hands-on Lab 17: Design an HCP Terraform Operating Model

Prerequisites

- HCP Terraform account or a documented offline simulation.
- A sample VCS repository.
- No production cloud credentials.

Implementation

1. Create a project and two workspaces representing dev and prod, or model them in a design document.
2. Define team permissions for readers, planners, maintainers, and production approvers.
3. Connect the dev workspace to VCS and document branch and working-directory triggers.
4. Design a variable-set hierarchy and prove which value wins for duplicate keys.
5. Add a run task or simulated external check with timeout and failure behavior.

6. Design an agent pool for a private network, including scaling, patching, token rotation, and outage response.
7. Run or simulate a plan-to-apply workflow and a failed-run incident.

Validation Evidence

- Access matrix follows least privilege.
- Workspace and project ownership are explicit.
- Run stages, external dependencies, and failure behavior are documented.
- Agent and state recovery procedures are testable.

Cleanup

- Destroy lab-managed resources or remove sample workspaces if appropriate.
- Revoke temporary tokens and disconnect test integrations.
- Retain sanitized operating-model diagrams and runbooks.

Interview Questions and Model Answers

1. What does an HCP Terraform workspace contain?

Configuration source and settings, variables, state, run history, permissions, and execution behavior for an infrastructure boundary.

2. Why use agents?

To execute Terraform within private or isolated networks while the agent polls outward, avoiding inbound access from the hosted service.

3. What is the risk of broad variable sets?

A change can affect many workspaces and expose credentials or alter behavior beyond the author's intended scope.

4. How should run-task outages be handled?

Define timeout, retry, observability, ownership, and fail-open or fail-closed behavior according to the control's risk.

Chapter Summary

- A managed platform centralizes controls but still requires architecture.
- Projects and workspaces should reflect state and ownership boundaries.
- Agents are production execution infrastructure.
- Permissions, variables, integrations, and recovery must be tested.

Chapter 18: Sensitive Data, Ephemeral Values, and Write-Only Arguments

Terraform state and plans often contain infrastructure identifiers and sensitive values. Marking a value sensitive only redacts normal display; it does not remove the value from state. Modern ephemeral values and provider write-only arguments can reduce persistence when used correctly.

Learning Objectives

- Differentiate sensitive, ephemeral, and write-only behavior.
- Use ephemeral variables, resources, and child-module outputs safely.
- Recognize provider support and version requirements.
- Design secret delivery, rotation, logging, and recovery around non-persistence.

Core Concepts

Sensitive value: The sensitive flag suppresses routine CLI display but the value can still be stored in state or plan artifacts and must be protected.

Ephemeral value: Terraform omits an ephemeral value from state and plan files. Ephemeral variables and child-module outputs can pass temporary runtime data through allowed contexts.

Ephemeral resource: An ephemeral block opens or obtains a temporary object for the operation and does not persist it as a managed resource in state.

Write-only argument: A provider-defined write-only resource argument accepts a temporary value for an operation and does not persist the value in state or plan files.

Version field: Providers may require a companion version or trigger argument so Terraform knows when a write-only secret should be updated.

Operational trade-off: Non-persistence reduces artifact exposure but makes replay, debugging, drift comparison, and recovery depend on the external secret system and rotation process.

Mechanism	Persisted in State?	Primary Use
sensitive = true	Usually yes	Redact routine display
ephemeral variable	No	Short-lived token or session input
ephemeral child output	No	Pass temporary value between modules
ephemeral resource	No	Obtain temporary credentials or secrets during run
write-only argument	No	Send secret to supported managed resource
external secret reference	Reference only	Runtime retrieval by workload or provider

Reference Configuration or Command Pattern

```
variable "session_token" {
  type    = string
  ephemeral = true
  sensitive = true
}

# Provider support is resource-specific.
# Illustrative pattern:
resource "example_database" "main" {
  name          = "orders"
  password_wo    = var.session_token
  password_wo_version = var.secret_version
}

variable "secret_version" {
  type = number
}
```

Production Checklist

- Terraform and provider versions explicitly support the selected feature.
- Secret values are not placed in source, tfvars, logs, saved plans, test fixtures, or debug output.
- External secret system availability and recovery are part of the run design.
- Rotation updates the correct write-only version or trigger and is tested end to end.
- State and plan access remain restricted because non-secret infrastructure data may still be sensitive.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Secret still appears in state	Only sensitive flag used; provider lacks write-only support	Rotate secret; restrict and sanitize artifacts; redesign delivery	New state contains no secret value where support exists
Apply cannot reproduce secret	Ephemeral input unavailable during operation	Restore external secret source and token exchange	Run obtains new temporary value
Rotation does not update resource	Version trigger unchanged	Increment or update documented companion argument	Remote service accepts new secret
Debug logs expose token	Verbose provider or shell output	Stop logging, rotate token, restrict artifacts, inspect pipeline	Secret scan and access review pass

Hands-on Lab 18: Prove Secret Non-Persistence

Prerequisites

- Terraform version supporting ephemeral values.
- A provider or simulation with ephemeral and write-only behavior.
- A disposable secret value.

Implementation

1. Create a sensitive normal variable and observe how it is redacted but represented in state using a safe dummy value.
2. Replace the flow with an ephemeral variable and supported ephemeral path.
3. Use a provider-supported write-only argument or a documented simulation with a companion version field.
4. Inspect plan JSON, state pull output, normal logs, and test artifacts for the dummy secret.
5. Rotate the secret and prove the intended resource update occurs.
6. Document recovery when the external secret broker is unavailable.

Validation Evidence

- The normal sensitive example demonstrates display redaction versus persistence.
- The ephemeral/write-only example omits the dummy value from state and plan.
- Rotation is controlled by an explicit version or trigger.
- No debug or CI artifact contains the dummy secret.

Cleanup

- Destroy all test resources.
- Delete state, plan, logs, and fixtures containing the dummy secret.
- Revoke or invalidate temporary values.

Interview Questions and Model Answers

1. Does sensitive = true keep a value out of state?

No. It primarily redacts display. State and plan storage still require protection.

2. What is an ephemeral variable?

A runtime input Terraform omits from state and plan files, available in supported ephemeral contexts.

3. What is a write-only argument?

A provider-defined resource argument used during an operation but not persisted by Terraform in state or plan.

4. What new operational dependency does non-persistence create?

The run must reliably obtain the temporary value again from an external identity or secret system, including during rotation and recovery.

Chapter Summary

- Sensitive display and non-persistence are different controls.
- Ephemeral values and write-only arguments reduce artifact exposure.
- Provider and Terraform version support must be verified.
- Secret rotation and recovery become external-system responsibilities.

Chapter 19: Terraform Stacks and Multi-Environment Orchestration

Traditional root modules and workspaces manage individual state boundaries well, but large platforms also need consistent orchestration across components and deployments. Terraform Stacks add a configuration layer for reusable components and repeated deployments managed as a unit.

Learning Objectives

- Explain Stack components, deployments, deployment groups, and orchestration.
- Compare Stacks with traditional workspaces and root-module pipelines.
- Design authentication, provider configuration, and component dependencies.
- Plan migration and rollout without turning the Stack into a new monolith.

Core Concepts

Component configuration: Stack component files define reusable infrastructure components sourced from Terraform modules and their provider requirements.

Deployment configuration: Deployment files define repeated deployments, inputs, provider configuration, identity, and rollout conditions.

Component lifecycle: Each component retains a distinct lifecycle and can depend on outputs from other components while participating in Stack-wide orchestration.

Deployment group: Groups organize deployment runs and can apply orchestration conditions or approval rules across a fleet.

Authentication: Stacks can obtain values from approved stores and use identity tokens for OIDC-based authentication, avoiding static credentials.

Choice: Use Stacks when repeated, coordinated deployments benefit from first-class orchestration. Use workspaces or pipelines when boundaries and rollout logic are already simple and clear.

Question	Workspace-Oriented Model	Stack-Oriented Model
Primary unit	One root module and state boundary	Collection of module-backed components and deployments
Repetition	Create multiple workspaces/pipelines	Define deployment configuration once and repeat
Orchestration	External pipeline or run triggers	First-class deployment and component orchestration
Migration complexity	Established patterns	Requires Stack design and controlled migration
Best fit	Independent infrastructure boundaries	Fleet or multi-environment platform with repeated topology

Reference Configuration or Command Pattern

```
# components.tfcomponent.hcl (illustrative)
component "network" {
  source = "./modules/network"
  inputs = {
    name = var.name
  }
  providers = {
    example = provider.example.this
  }
}

component "service" {
  source = "./modules/service"
  inputs = {
    network_id = component.network.id
  }
  providers = {
    example = provider.example.this
  }
}
```

Production Checklist

- Each component maps to a coherent ownership and failure boundary.
- Deployment identity is short-lived and scoped.
- Dependencies are explicit and avoid circular orchestration.
- Rollout groups, approvals, conditions, and rollback are tested with canaries.
- Migration retains state history, contracts, and a documented fallback path.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Stack validation fails	Component/provider/input contract inconsistent	Validate component and deployment files separately; inspect exact interface	terraform stacks validate succeeds
Deployment blocked unexpectedly	Condition, approval, or dependency not satisfied	Inspect deployment group rule and upstream outputs	Canary proceeds under documented condition
Component change affects all deployments	Unscoped rollout or shared version update	Pin versions; canary deployment groups; pause fleet	Only intended group receives candidate
Migration loses history or binding	Beta/legacy assumptions or incomplete migration plan	Use current migration tooling and official guidance; back up evidence	Deployment plan matches existing infrastructure

Hands-on Lab 19: Design a Multi-Environment Terraform Stack

Prerequisites

- Terraform CLI with Stacks support and HCP Terraform access, or an offline design simulation.

- Two reusable modules.
- No production credentials.

Implementation

1. Define network and service components in .tfcomponent.hcl files.
2. Define dev, staging, and prod deployments with different validated inputs.
3. Use an identity-token design or documented federation flow for provider authentication.
4. Create deployment groups for canary and production rollout with explicit conditions.
5. Validate the Stack and run or simulate a change promoted from dev to production.
6. Inject a failing service component and demonstrate containment, pause, and recovery.
7. Compare the design with equivalent workspace pipelines and record the decision.

Validation Evidence

- Component and deployment configuration validate successfully.
- Each deployment has explicit identity, inputs, and rollout group.
- A failed component does not create an uncontrolled fleet rollout.
- The architecture decision explains why Stacks are or are not appropriate.

Cleanup

- Destroy test deployments or remove simulated Stack resources.
- Revoke temporary identities and delete local Stack artifacts containing metadata.
- Retain sanitized component, deployment, and migration documentation.

Interview Questions and Model Answers

1. What problem do Terraform Stacks solve?

They provide a higher-level way to compose module-backed components and deploy the same topology across multiple environments with coordinated lifecycle and rollout controls.

2. Are Stacks a replacement for modules?

No. Stack components source Terraform modules; modules remain the reusable infrastructure implementation unit.

3. When should a team stay with workspaces?

When infrastructure boundaries are independent, repetition is limited, and existing pipelines already provide clear, reliable orchestration.

4. How should Stack authentication work?

Prefer short-lived federated identity, such as OIDC identity tokens, with provider permissions scoped to the deployment.

Chapter Summary

- Stacks add component and deployment orchestration above modules.
- They are useful for repeated multi-environment or fleet topologies.
- Component boundaries must still preserve ownership and failure containment.
- Migration, identity, canary rollout, and rollback require explicit design.

Chapter 20: Enterprise Terraform Operations and Part B Capstone

Production Terraform is a socio-technical operating system: repositories, modules, state, identities, runners, policies, tests, approvals, service validation, drift response, incident command, and recovery must work together under failure.

Learning Objectives

- Design an end-to-end enterprise Terraform delivery platform.
- Integrate tests, policy, short-lived identity, plans, approvals, and service validation.
- Operate drift, partial apply, provider outage, lock contention, and state recovery.
- Demonstrate a complete evidence-backed capstone.

Core Concepts

Golden workflow: A standard path moves reviewed code through static checks, tests, plan, security and policy, approval, apply, acceptance, and evidence retention.

Run identity: Each run uses short-lived, attributable identity scoped to its state and environment. Human credentials are not automation credentials.

Concurrency: One active writer per state, explicit queue ownership, cancellation rules, stale-plan invalidation, and safe lock recovery are mandatory.

Drift program: Scheduled detection routes drift to accept, reverse, ignore, import, or transfer-ownership decisions with service-owner approval.

Incident command: Failed applies require change freeze, evidence preservation, incident roles, provider/API diagnosis, state safety, functional validation, and documented recovery.

Recovery evidence: State backup is not enough. Teams must prove restoration from a clean runner and produce a no-change plan plus service acceptance.

Run Stage	Control	Evidence
Source	Protected branch and reviewed module versions	Commit, reviewers, signatures
Test	Static, contract, integration, upgrade	Test report and cleanup
Plan	Pinned inputs, lock file, state, identity	Saved plan digest and summary
Govern	Security, cost, policy, exception	Decision and expiry
Apply	Serialized protected execution	Run log and state version
Validate	Functional, security, reliability	Acceptance results and rollback decision
Operate	Drift, monitoring, recovery	Tickets, drills, SLOs, postmortems

Reference Configuration or Command Pattern

```
# Enterprise quality gate sequence (illustrative)
terraform fmt -check -recursive
terraform init -backend=false
terraform validate
terraform test

# Protected environment job:
terraform init -reconfigure -backend-config=backend.hcl
terraform plan -out=tfplan
terraform show -json tfplan > tfplan.json
# security, cost, policy, approval
terraform apply tfplan
# service acceptance, evidence, cleanup of sensitive artifacts
```

Production Checklist

- The exact reviewed commit, lock file, inputs, policy version, and state context flow from plan to apply.
- Production applies are serialized, approved, attributable, and use short-lived identity.
- State restore, provider outage, lock recovery, and partial-apply runbooks are tested.
- Every policy and test failure has an owner and remediation route.
- Service-level acceptance and rollback decision occur after infrastructure apply.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Apply partially succeeds then provider errors	Remote API timeout or provider crash	Freeze writes; preserve state and logs; refresh/plan; validate remote reality	Recovery plan contains only intended reconciliation
Approved plan becomes stale	State changed after approval	Discard and regenerate; never force old plan through	New plan reflects latest state serial and commit
State unavailable	Backend outage, auth, DNS, or corruption	Stop applies; invoke backend incident and restore procedure	Clean runner produces no-change plan
Drift flood overwhelms team	Unowned scheduled detection	Triage by severity and ownership; suppress only with decision	Backlog has owner, SLA, and trend

Hands-on Lab 20: Enterprise Terraform Platform Capstone

Prerequisites

- Completed Chapters 11-19.
- Git repository and CI system or documented local simulation.
- HCP Terraform/Enterprise optional; provider-neutral execution is acceptable.

Implementation

1. Design a platform with separate foundation, shared-service, and workload states plus versioned contracts.
2. Publish a tested reusable module and consume it in dev, staging, and prod.

3. Implement static checks, terraform test, plan summary, security scan placeholder, policy pack, approval, apply, and acceptance stages.
4. Use provider aliases or simulated target contexts with short-lived identity design.
5. Demonstrate sensitive normal values versus ephemeral/write-only non-persistence.
6. Model the deployment through workspaces or a Terraform Stack and justify the choice.
7. Inject manual drift, an invalid policy change, a stale plan, a lock conflict, and a partial apply; execute the runbooks.
8. Restore state and execution from a clean working environment and produce a no-change plan.
9. Create an evidence pack with architecture, ADRs, test results, policy decisions, run history, validation, cleanup, and lessons.

Validation Evidence

- State and ownership boundaries are explicit and independently recoverable.
- Module tests, policy tests, and upgrade tests pass.
- Plan and apply inputs are identical and attributable.
- Secret values are absent from protected artifacts where ephemeral/write-only support is used.
- Failure drills produce controlled recovery and a final no-change plan.

Cleanup

- Destroy all capstone-managed resources.
- Remove state, plan JSON, saved plans, debug logs, temporary credentials, and test fixtures from shared locations.
- Retain only sanitized portfolio evidence, diagrams, code, and runbooks.

Interview Questions and Model Answers

1. What is the difference between a Terraform pipeline and a Terraform operating model?

A pipeline is execution automation. The operating model also defines architecture, ownership, identity, access, policy, testing, approvals, drift, incident response, recovery, and service acceptance.

2. What should happen after a partial apply?

Stop concurrent changes, preserve evidence, inspect state and remote systems, regenerate a plan, reconcile intentionally, validate the service, and document the incident.

3. How do you prove state recovery works?

Restore access or a state version, initialize from a clean runner with pinned dependencies and identity, refresh or plan safely, and demonstrate the expected no-change or controlled recovery plan.

4. What is the strongest indicator of Terraform maturity?

Another authorized engineer can reproduce, review, recover, and safely operate the system using documented controls and evidence, not tribal knowledge.

Chapter Summary

- Enterprise Terraform combines architecture, contracts, tests, policy, identity, controlled runs, validation, drift, and recovery.
- A reviewed plan is valid only for its exact execution context.
- Failures are expected and rehearsed.
- The capstone demonstrates platform-level Terraform engineering.

Part B Advanced Terraform Command Reference

Purpose	Command or Pattern	Operational Note
Run tests	terraform test	Protect test state and ensure cleanup.
Filter tests	terraform test -filter=tests/name.tfstate.hcl	Use focused reproduction before full suite.
JSON plan	terraform show -json tfplan	May contain sensitive values; restrict and delete.
Expression console	terraform console	Use pinned configuration and variables.
Dependency graph	terraform graph	Sanitize diagrams before sharing.
Provider schema	terraform providers schema -json	Useful for policy and tooling; version-specific.
Provider inventory	terraform providers	Confirms requirements and state bindings.
Replace provider	terraform state replace-provider FROM TO	Back up and peer-review before provider-source migration.
Workspace list	terraform workspace list	CLI workspaces are state instances, not complete isolation by themselves.
HCP login	terraform login	Use supported credential storage; never commit token.
Stacks initialize	terraform stacks init	Initializes Stack configuration and dependencies.
Stacks validate	terraform stacks validate	Validates component and deployment definitions.
Stacks commands	terraform stacks --help	Confirm exact commands for installed version.
State backup	terraform state pull > backup.tfstate	Encrypt, restrict, and delete after recovery window.
Lock recovery	terraform force-unlock LOCK_ID	Only after proving original writer is absent.

Advanced Terraform Production Readiness Checklist

Architecture

- [] State boundaries follow ownership, lifecycle, sensitivity, coupling, and recovery.
- [] Cross-state contracts are narrow, versioned, and tested.
- [] Provider targets and aliases are explicit.

- ☐ Workspaces or Stacks are chosen through an ADR.

Module Product

- ☐ Contract is typed, documented, and semantically versioned.
- ☐ Examples, contract tests, integration tests, and upgrade tests pass.
- ☐ Releases are immutable and traceable.
- ☐ Deprecation and security-response policies exist.

Governance

- ☐ Policies have tests, owners, severity, remediation, and rollout plans.
- ☐ Exceptions include compensating control and expiry.
- ☐ External run tasks define timeout and fail behavior.
- ☐ Policy and test metrics are reviewed.

Identity and Secrets

- ☐ Automation uses short-lived least-privilege identity.
- ☐ Provider mappings match trust boundaries.
- ☐ Sensitive artifacts are restricted and retained minimally.
- ☐ Ephemeral/write-only flows are tested where supported.

Operations

- ☐ Applies are serialized and stale plans are discarded.
- ☐ Drift has an owner and decision workflow.
- ☐ State restore, lock recovery, partial apply, and provider outage are rehearsed.
- ☐ Service acceptance and rollback follow apply.

Ten Additional Advanced Terraform Projects

1. State Decomposition Factory

Create a repeatable assessment and migration toolkit for splitting monolithic states without recreation.

- Deliverables: code, tests, architecture diagram, threat model, runbooks, validation evidence, cleanup proof, and lessons learned.

2. Provider Topology Harness

Test multi-account and multi-region provider alias mappings using mocks and disposable integration targets.

- Deliverables: code, tests, architecture diagram, threat model, runbooks, validation evidence, cleanup proof, and lessons learned.

3. Module Release Platform

Automate semantic release, documentation, tests, registry publication, provenance, and consumer upgrade reports.

- Deliverables: code, tests, architecture diagram, threat model, runbooks, validation evidence, cleanup proof, and lessons learned.

4. Upgrade Safety Lab

Maintain old-state fixtures and prove candidate Terraform, provider, and module upgrades are compatible.

- Deliverables: code, tests, architecture diagram, threat model, runbooks, validation evidence, cleanup proof, and lessons learned.

5. Policy Control Library

Build tested security, reliability, cost, metadata, and destructive-change policies with expiring exceptions.

- Deliverables: code, tests, architecture diagram, threat model, runbooks, validation evidence, cleanup proof, and lessons learned.

6. HCP Terraform Operating Model

Design projects, workspaces, teams, variable sets, agents, run tasks, policy sets, and recovery.

- Deliverables: code, tests, architecture diagram, threat model, runbooks, validation evidence, cleanup proof, and lessons learned.

7. Secret Non-Persistence Demonstrator

Compare sensitive, ephemeral, external-reference, and write-only patterns with artifact scans.

- Deliverables: code, tests, architecture diagram, threat model, runbooks, validation evidence, cleanup proof, and lessons learned.

8. Terraform Stack Fleet

Deploy a component topology to multiple environments with canary groups and federated identity.

- Deliverables: code, tests, architecture diagram, threat model, runbooks, validation evidence, cleanup proof, and lessons learned.

9. Drift Response Service

Schedule detection, classify drift, route decisions, measure age, and validate reconciliation.

- Deliverables: code, tests, architecture diagram, threat model, runbooks, validation evidence, cleanup proof, and lessons learned.

10. Terraform Incident Simulator

Inject stale plans, locks, provider failures, partial applies, state loss, and policy outages with runbooks.

- Deliverables: code, tests, architecture diagram, threat model, runbooks, validation evidence, cleanup proof, and lessons learned.

Senior Terraform Interview Drill Bank

1. How would you split a large Terraform state?

Analyze ownership, lifecycle, coupling, sensitivity, change frequency, identity, and recovery. Freeze writes, back up state, migrate bindings, compare plans, validate services, and preserve rollback evidence.

2. How do you test a Terraform module?

Use static checks, plan-based contract tests, negative tests, provider mocks, limited disposable integration tests, upgrade fixtures, policy tests, and service acceptance.

3. How do you prevent deployment to the wrong account?

Make provider aliases and target metadata explicit, map providers at module boundaries, use scoped short-lived identities, remove accidental defaults, and test mappings.

4. How do sensitive and ephemeral values differ?

Sensitive redacts normal display but may persist. Ephemeral values are omitted from plan and state in supported contexts.

5. When are Terraform Stacks useful?

When module-backed components must be deployed repeatedly across environments or fleets with first-class orchestration, rollout groups, and coordinated lifecycle.

6. How do you recover from a partial apply?

Freeze changes, preserve evidence, inspect state and remote reality, regenerate a plan, reconcile intentionally, validate the service, and publish a postmortem.

7. What is policy-as-code maturity?

Tested, versioned, scoped controls with owners, remediation, rollout, metrics, and expiring exceptions - not just blocking rules.

8. What is wrong with one automation credential?

It creates excessive blast radius, poor attribution, difficult rotation, and cross-environment compromise. Use short-lived identity scoped to state and environment.

9. How do you prove state recovery?

Restore from version or backup, initialize a clean runner with pinned dependencies and correct identity, then produce and validate the expected plan.

10. What is a mature Terraform platform?

A reproducible operating model where architecture, modules, tests, policy, identity, runs, validation, drift, incidents, and recovery are evidence-backed and transferable between engineers.

Glossary

Term	Definition
Agent pool	A group of self-hosted HCP Terraform or Terraform Enterprise agents used to execute runs in controlled networks.
Check block	A configuration assertion used to evaluate infrastructure or data conditions during operations and continuous validation.
Component	In Terraform Stacks, a module-backed infrastructure unit with inputs, providers, outputs, and lifecycle.
Deployment	A configured instance of a Stack topology for an environment, region, account, tenant, or other target.
Deployment group	A grouping used to coordinate Stack deployment runs and orchestration conditions.
Ephemeral value	A Terraform runtime value omitted from state and plan artifacts in supported contexts.
Expected failure	A Terraform test declaration that a particular variable, condition, or check should fail.
Mock provider	A test provider substitute that generates or overrides values without calling the remote API.
Policy set	A versioned collection of policies attached to selected Terraform run scopes.

Term	Definition
Postcondition	An assertion evaluated after an operation to verify a resulting attribute or invariant.
Precondition	An assertion evaluated before an operation to protect an assumption.
Project	An HCP Terraform grouping and access boundary for workspaces and Stacks.
Run task	An external integration invoked at a Terraform run stage to evaluate or enrich the run.
Semantic versioning	A release model communicating compatible fixes, compatible features, and breaking changes through version numbers.
Stack	A Terraform configuration layer that composes module-backed components and manages repeated deployments.
Write-only argument	A provider-defined resource argument accepted during an operation but omitted from Terraform state and plans.

Official Terraform Documentation Index

Reference	Official URL
Terraform language	https://developer.hashicorp.com/terraform/language
Module development	https://developer.hashicorp.com/terraform/language/modules/develop
Terraform tests	https://developer.hashicorp.com/terraform/language/tests
Provider mocking	https://developer.hashicorp.com/terraform/language/tests/mocking
Custom conditions	https://developer.hashicorp.com/terraform/language/expressions/custom-conditions
Sensitive data management	https://developer.hashicorp.com/terraform/language/manage-sensitive-data
Ephemeral values	https://developer.hashicorp.com/terraform/language/manage-sensitive-data/ephemeral
Write-only arguments	https://developer.hashicorp.com/terraform/language/manage-sensitive-data/write-only
Terraform Stacks overview	https://developer.hashicorp.com/terraform/language/stacks
Design a Stack	https://developer.hashicorp.com/terraform/language/stacks/design

Reference	Official URL
Stack configuration files	https://developer.hashicorp.com/terraform/language/files/stack
Stacks authentication	https://developer.hashicorp.com/terraform/language/stacks/deploy/authenticate
HCP Terraform overview	https://developer.hashicorp.com/terraform/cloud-docs/overview
HCP Terraform projects	https://developer.hashicorp.com/terraform/cloud-docs/projects
HCP Terraform workspaces	https://developer.hashicorp.com/terraform/cloud-docs/workspaces
HCP Terraform agents	https://developer.hashicorp.com/terraform/cloud-docs/agents
Policy enforcement	https://developer.hashicorp.com/terraform/cloud-docs/policy-enforcement
Run tasks	https://developer.hashicorp.com/terraform/cloud-docs/workspaces/settings/run-tasks
Permissions	https://developer.hashicorp.com/terraform/cloud-docs/users-teams-organizations/permissions
Terraform CLI commands	https://developer.hashicorp.com/terraform/cli/commands

Terraform CLI, providers, HCP Terraform, Terraform Enterprise, Stacks, policies, pricing, and certification objectives evolve. Verify current official documentation, release notes, provider schemas, and licensing before production adoption.

Volume 9B Completion

Part B completes advanced Terraform engineering. The next section begins Ansible as the configuration-management and orchestration complement to Terraform.

Next part: Volume 9C - Ansible Foundations, Inventory, Ad Hoc Operations, Playbooks, Variables, Templates, Handlers, and Configuration Automation.